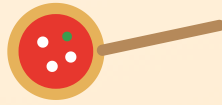


FORNO (GPU 80GB)



 estilo Use a Cabeça

A Pizzaria do Seu Nemotron

O job de treino, **linha por linha** — explicado como uma noite na pizzeria



Seu Nemotron

o chef genial (modelo 30B)



Molho da Vovó

a base que já é boa
(adapter 086 = 0.86)



O Forno

a GPU. Tamanho importa!
(80GB)



Massa Pronta

o *wheel* (instala em 10s)



O Inspetor

o *watchdog* (desliga se
pegar fogo)



A Câmera

o *live-log* (transmite pro
Claude)

1 🍄 Buscar ingredientes e montar a cozinha (deps)

🛒 Numa frase: vai ao mercado, **troca a batedeira pelo modelo certo** e pega a **massa pronta** — pra não perder 30 min sovando.

```
import os, subprocess, sys # ← pega 3 ferramentas do Python
print('[1/5] clone repo branch', flush=True) # ← avisa em voz alta
```

- `import os, subprocess, sys` → 3 ajudantes: mexer em pastas, rodar comandos, saber qual Python.
- `flush=True` → força a frase a **aparecer na hora** (sem ele você fica no escuro).

```
subprocess.run(['git', 'clone', ... , '/content/kg1'], check=True) # ← baixa a apostila
```

- `git clone` → baixa o repositório (a **apostila de receitas**).
- `--depth 1` → só a última página (download rápido). `--branch` → a apostila certa.
- `check=True` → 🔑 **se a apostila não chegar, a cozinha NEM ABRE** (erro aqui).

🗿 Não existe pergunta idiota

— Por que `check=True` só aqui? Sem apostila não dá pra fazer nada (tem que parar). Um `pip` que reclama de bobeira não derruba a cozinha → por isso os installs usam `check=False`.

```
subprocess.run([ ... 'torch==2.10.0', '--index-url', '... /cu126']) # ← TROCA a batedeira
```



- **O PLOT TWIST:** o Colab veio com a batedeira **2.11** (nova demais). A **massa pronta** só encaixa na **2.10** → a gente troca de propósito.
- `--index-url ... /cu126` → 🔑 compra na loja certa, que entrega a batedeira **com motor (CUDA)**.

⚠️ CUIDADO!

Loja errada (PyPI padrão) entrega batedeira **sem motor (CPU)** — linda, mas não bate nada. O `--index-url cu126` evita o desastre.

```
url = f" ... mamba_ssm-2.3.1+cu12torch2.10cxx11abiTRUE-cp312 ... whl"
if pip install --no-deps url FALHOU: compila do source (~30min) # ← plano B
```

- 📦 Pega a **massa pronta (~10s!)** montando o endereço pela etiqueta da batedeira.
- 🔑 **Plano B automático:** se a massa pronta não existir, **sova do zero (~30min)**. Nunca quebra — no pior caso fica lento.

🗿 Não existe pergunta idiota

— Se tem plano B, por que ligar pra massa pronta? Porque o plano B custa **30 min** por treino; a massa pronta, **10 s**. Multiplica por N treinos... é MUITA pizza esperando. 🍕🕒

```
try: import mamba_ssm, causal_conv1d # ← prova uma colherada
except: raise RuntimeError( ... ) # ← estragou? fecha a cozinha
```

```
print('DEPS OK')
```

← mise en place pronto!

- 🍷 **Prova uma colherada:** se massa/molho estragaram, **para AGORA** com recado. Senão: `DEPS OK`.

📄 **Resumo:** apostila baixada → bateadeira 2.10 (com motor) → massa pronta (10s) → molho (5min) → provou → DEPS OK.

2 🔑 A chave do depósito (HF token)



depósito (HF)

```
v = userdata.get('HF_KEY') # ← pega a chave do cofre do Colab
assert os.environ.get('HF_TOKEN'), 'Defina HF_KEY ...' # ← sem chave, não entra
```

- 🔑 Busca a **chave do depósito** (o segredo `HF_KEY` do Colab). Tenta 3 nomes possíveis.
- `assert` → 🔑 **porteiro:** sem chave, não pega o Molho da Vovó nem guarda a pizza → para com recado.

⚠️ CUIDADO!

A chave precisa ser de **escrita** (pra GUARDAR a pizza no HF, não só olhar).

3 🔥 Medir o forno (GPU-guard)



40GB 🍷 não aquece



>forno → 🔥 OOM



80GB 🍷 aquece

```
vram = total_memory / 1e9
assert vram ≥ 70, '40GB → RECONECTE p/ 80GB (vai dar OOM)' # ← forno pequeno? para!
```

- 🔑 A pizza família (30B) precisa de ~**60GB**. Forninho de 40GB **queima (OOM)**. A gente **mede ANTES** e para com recado claro.

🔥 Esquente os neurônios

Por que medir o forno ANTES? Carregar o 30B leva ~10min. Descobrir "forno pequeno" DEPOIS = 10min jogados fora + crash feio. Medir antes = **1 segundo**. ⌚

⚠️ Isto é um ERRO BOM

Quando ele para com `AssertionError: VRAM=42GB`, NÃO é bug — é o **detector de fumaça** apitando antes do incêndio. Solução: reconectar até vir 80GB.

4 A ficha da masterclass (config)

```
MODE = 'SMOKE'      # ← o botão: SMOKE (ensaio 8 passos) ou REAL (160 passos)
INIT_ADAPTER = '...submit086'; REQUIRE_INIT_ADAPTER='1' # ← parte do Molho da Vovó
LEARNING_RATE = 5e-6 → 1e-6 # ← ritmo: firme no início, suave no fim
```

- **MODE** = o único botão. **SMOKE** = aula experimental (barato). **REAL** = masterclass.
- 🔑 **INIT_ADAPTER 086** → começa do **Molho da Vovó** (não recomeça do zero) — e **REQUIRE** exige.
- **Ir 5e-6→1e-6** → começa firme, suaviza pra **não estragar** o que o chef já sabia.

⚠ CUIDADO! (a conta que quase deu errado)

Pra ter **160 passos** precisa de **6 turmas** (`NUM_EPOCHS=6`). Com 1 turma daria **31**! Conta: `ceil(979/32)=31` · `31×6=186` → corta em 160. ✓

🗿 Não existe pergunta idiota

— **Por que SMOKE exige o stream e REAL não?** No ensaio a gente QUER que falhe alto se a câmera não funcionar. No treino de 2-3h, um tropeço de upload **não pode** derrubar tudo.

5 Cozinhar com câmera e Inspetor na porta (train)



+ vigia →



👮 Inspetor

```
os.environ['KG1_LIVE_LOG_HF_REPO'] = '.../kg1-live-logs' # ← liga a câmera
os.environ['KG1_WATCHDOG_STALE_SECONDS'] = '2700' # ← Inspetor: 45min parado = fecha
r = subprocess.run([... runner.py, '--', python, train.py]) # ← AÇÃO!
print('RETURN_CODE=', r.returncode) # ← a nota do prato
```

- 🔑 **Câmera ao vivo**: tudo vai pro HF pro crítico (Claude) ver de casa. `RUN_ID` = nome do episódio.
- **Inspetor (watchdog)**: se a cozinha ficar 45min muda, ou pegar fogo (NaN/OOM), **desliga**.
- `'--'` separa o **vigia** (runner) do **vigiado** (o chef/train). `RETURN_CODE=0` = pizza perfeita.

💡 Esquente os neurônios

O que o `--` faz? Separa o "vigia" do "vigiado". Tudo depois do `--` é o comando que o Inspetor roda e filma. 🎬

? O teste do garçom (responda sem espiar)

1. O que `flush=True` faz? → **força o print a aparecer na hora**
2. Por que trocar pra bateadeira 2.10? → **a massa pronta só encaixa nela → poupa 30min**
3. Por que `--index-url cu126`? → **garante a bateadeira COM motor (CUDA), não CPU**
4. Por que `NUM_EPOCHS=6`? → **$31 \times 6 = 186$ → corta em 160 (1 turma daria só 31)**
5. `RETURN_CODE=0` significa? → **treino terminou bem; a pizza saiu**

Resumo do capítulo

Cél 1: ingredientes + bateadeira 2.10 (com motor) + massa pronta (10s) + molho (5min).

Cél 2: a chave do depósito (token de escrita).

Cél 3: medir o forno (80GB cabe a pizza família 30B; 40GB = OOM, o guard para).

Cél 4: a ficha (SMOKE/REAL, Molho da Vovó 086, ritmo 5e-6 → 1e-6, 160 passos).

Cél 5: cozinhar com câmera ao vivo + Inspetor (45min stall / NaN / OOM = fecha). Nota no fim.

Resposta do "Afie o lápis"

`transformers==4.57.6` é pinado porque (a) entende o chef NemotronH e (b) é aceito pelo motor de avaliação (vLLM exclui a 5.x). **A mais nova nem sempre é a melhor — a melhor é a que ENCAIXA.** ✨